Integrante 3 — Interfaz MVC de personapi-dotnet: investigación y plan

Oct 20, 2026 · @samuel

Portada

Pontificia Universidad Javeriana Facultad de Ingeniería — Ingeniería de Sistemas Arquitectura de Software

Laboratorio 1: Monolito con patrones MVC y DAO — personapi-dotnet Documento de investigación y plan del integrante 3: interfaz MVC

Responsable: Samuel Repositorio del equipo: github.com/sara-munoz-5/personapi-dotnet Bogotá, octubre de 2026

Mi parte

Me toca construir la interfaz web: 4 controladores MVC y 20 vistas Razor para el CRUD de persona, profesión, estudios y teléfono, sobre .NET 10 y ASP.NET Core MVC.

Qué entrego

- PersonaController, ProfesionController, EstudioController y TelefonoController.
- Cinco vistas por entidad: Index (lista), Details (detalle), Create (crear), Edit (editar) y Delete (confirmar borrado).
- Listas desplegables en los formularios de Estudio (persona y profesión) y de Teléfono (dueño).
- Validaciones de formularios y mensajes de error claros.
- Menú del layout con las cuatro entidades y un enlace a Swagger.
- Capturas de cada vista para el documento final.

De qué dependo

- Del integrante 1: las entidades generadas por scaffolding, las interfaces DAO (IPersonaRepository y las demás) y sus repositorios registrados en Program.cs.
- De lo que acordemos los tres sobre las firmas de esas interfaces. Si una firma cambia, mis controladores dejan de compilar.

Qué no me toca

- La base de datos, los scripts SQL ni los repositorios (integrante 1).
- Los controladores API, Swagger ni los endpoints /api/... (integrante 2).
- Mis controladores no llaman a la API: usan los mismos repositorios que ella, por inyección de dependencias.

Organización del equipo

El trabajo se reparte por capa. La arquitectura, las pruebas finales y las conclusiones se hacen entre los tres.

Responsable	Bloque	Qué entrega	Carpetas
Integrante 1	Persistencia	Scripts DDL y DML, base persona_db, scaffolding (entidades y PersonaDbContext), cadena de conexión, interfaces DAO, repositorios y su registro en Program.cs	scripts/, Models/Entities/, Interfaces/, Repository/
Integrante 2	Servicios REST	4 controladores API con los 20 endpoints, códigos HTTP, Swashbuckle y Swagger UI	Controllers/Api/
Integrante 3 (Samuel)	**Interfaz MVC**	**4 controladores MVC, 20 vistas Razor, listas desplegables, validaciones y menú del layout**	**Controllers/, Views/, wwwroot/**
Los tres	Colaborativo	Arquitectura y firmas de las interfaces, pruebas finales, conclusiones y lecciones aprendidas, README y TAG	docs/, README.md

Orden de trabajo

1. Los tres acuerdan la arquitectura y las firmas de las interfaces DAO: métodos, parámetros y tipos de retorno. Es el contrato que une las tres partes.
2. El integrante 1 sube a main la base: proyecto, scaffolding, interfaces y conexión. Los integrantes 2 y 3 parten de ahí.
3. Los integrantes 2 y 3 trabajan en paralelo sobre las mismas interfaces.
4. El integrante 1 termina los repositorios; mientras tanto, 2 y 3 prueban con los datos del DML.
5. Los tres integran, hacen las pruebas finales, escriben las conclusiones y crean el TAG.

Reglas de git

- main siempre compila; nadie sube directo a main después de la base del integrante 1.
- Una rama por integrante: feature/persistencia, feature/api-rest y feature/mvc (la mía).
- Se integra con pull request y otro integrante lo revisa.
- Si alguien necesita cambiar una interfaz DAO, lo avisa a los otros dos antes.
- Commits pequeños con mensaje en español ("Agrega vistas CRUD de teléfono").
- No se suben bin, obj ni .vs.
- El TAG se crea solo cuando main está probado de punta a punta por los tres.

Firmas DAO que necesito acordar

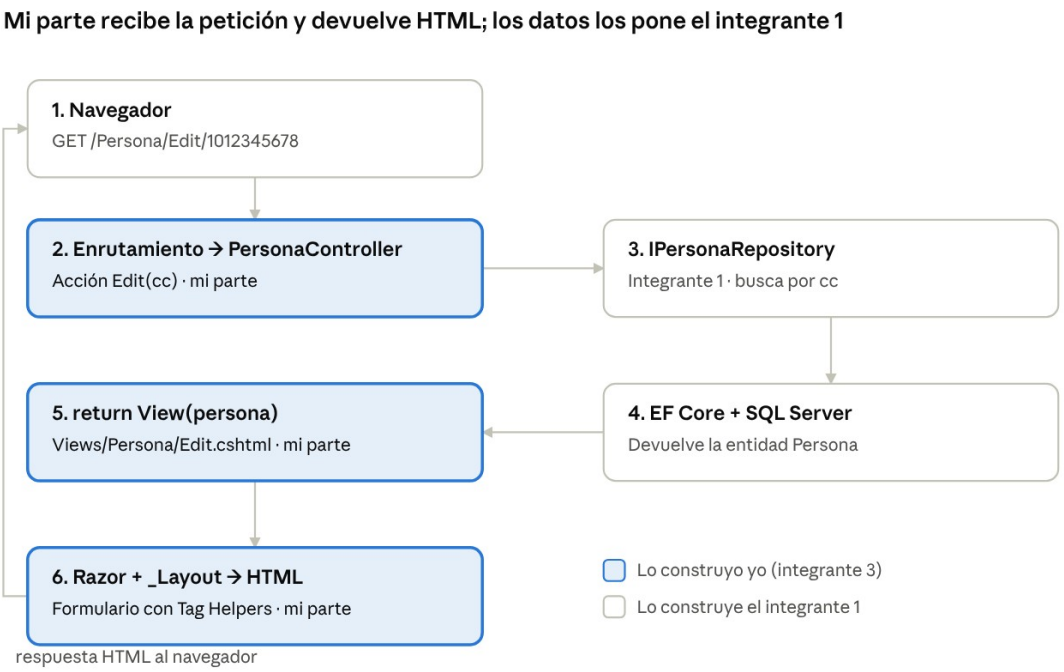
Para cada entidad necesito cinco operaciones; esta es la propuesta que llevo a la reunión del paso 1.

Operación	Qué recibe	Qué devuelve	La uso en
Listar todos	nada	lista de la entidad, con sus relaciones cuando aplica	Index
Buscar por llave	cc, id, num, o (idProf, ccPer) en Estudio	la entidad o nulo si no existe	Details, Edit (GET), Delete (GET)
Crear	la entidad	nada; lanza error si la llave está repetida	Create (POST)
Actualizar	la entidad	nada	Edit (POST)
Eliminar	la llave	nada	Delete (POST)

Todas asíncronas (devuelven Task) para no bloquear el servidor mientras espera a SQL Server. Para Index de Estudio y Teléfono pido que "listar todos" incluya la persona y la profesión relacionadas, así muestro nombres en lugar de números.

Cómo funciona MVC en ASP.NET Core 10

En MVC, el controlador recibe la petición, pide los datos al modelo y elige la vista que los muestra. Mi parte son el controlador y la vista; el modelo y su acceso a datos son del integrante 1.



recorrido de una petición MVC · 6 pasos

Al guardar el formulario el recorrido se repite con un POST: el controlador valida, llama a Actualizar del repositorio y redirige a la lista.

Controladores

Un controlador MVC es una clase que hereda de Controller; cada método público es una acción que responde a una URL y casi siempre devuelve una vista.

Enrutamiento por convención

La plantilla MVC registra la ruta {controller=Home}/{action=Index}/{id?}. Así, /Persona/Edit/123 llama a la acción Edit de PersonaController con id = 123, sin configurar nada más. Si falta el controlador o la acción, se usan Home e Index. El {id?} es opcional.

Estudio tiene dos valores de llave y no cabe en {id}. Los paso como parámetros adicionales: /Estudio/Edit?idProf=3&ccPer=1012345678. El enlazador de modelos los asigna por nombre a los parámetros de la acción.

Resultados que devuelve una acción

Resultado	Cuándo lo uso
View(modelo)	Mostrar una vista con datos: Index, Details, formularios
RedirectToAction(nameof(Index))	Después de crear, editar o borrar con éxito
NotFound()	La llave no existe en Details, Edit o Delete
View(modelo) otra vez, con errores	El formulario llegó con datos inválidos

Inyección de dependencias

El controlador no crea el repositorio: lo recibe en el constructor como interfaz (IPersonaRepository) y ASP.NET Core le entrega la implementación registrada en Program.cs. El DbContext se registra con vida Scoped (uno por petición) y los repositorios deben tener la misma vida para compartirlo. Esto es lo que hace que la capa de presentación no dependa de EF Core.

Acciones asíncronas

Como los repositorios devuelven Task, las acciones son async y esperan (await) cada llamada. Mientras SQL Server responde, el hilo atiende otras peticiones.

GET y POST por acción

Create, Edit y Delete tienen dos versiones con el mismo nombre: la GET muestra el formulario y la POST, marcada con [HttpPost], lo procesa. Para Delete, la POST se llama DeleteConfirmed y lleva [ActionName("Delete")] porque tendría la misma firma que la GET.

Patrón Post/Redirect/Get (PRG)

Después de un POST exitoso no devuelvo una vista: redirijo a Index. Si el usuario recarga la página, el navegador repite el GET de Index y no el POST, así no se crean registros duplicados. Para avisar "Persona creada" uso TempData, que sobrevive exactamente a una redirección.

Vistas Razor

Una vista es un archivo .cshtml que mezcla HTML con C# mediante la sintaxis Razor (@). El servidor la convierte en HTML y la envía al navegador.

Cómo se encuentra la vista

Si PersonaController.Edit devuelve View() sin nombre, ASP.NET Core busca Views/Persona/Edit.cshtml y, si no existe, Views/Shared/Edit.cshtml. Por eso la carpeta se llama igual que el controlador sin la palabra "Controller".

Archivos compartidos

Archivo	Para qué sirve
Views/Shared/_Layout.cshtml	Plantilla común: encabezado, menú, pie y Bootstrap. Cada vista se dibuja dentro de su RenderBody()
Views/_ViewStart.cshtml	Indica que todas las vistas usan _Layout
Views/_ViewImports.cshtml	Importa espacios de nombres (Models.Entities) y activa los Tag Helpers para todas las vistas
Views/Shared/_ValidationScriptsPartial.cshtml	Carga jQuery Validation para validar en el navegador
Views/Shared/Error.cshtml	Página de error genérica

Vistas fuertemente tipadas

Cada vista declara su modelo con @model: Index recibe una lista de Persona; Details, Create, Edit y Delete reciben una Persona. Así Visual Studio autocompleta las propiedades y un nombre mal escrito falla al compilar, no en tiempo de ejecución.

Tag Helpers

Son atributos asp-* que el servidor convierte en HTML correcto. Prefiero los Tag Helpers sobre los HTML Helpers (@Html.TextBoxFor) porque el marcado se lee como HTML normal.

Tag Helper	Qué hace	Ejemplo de uso en mis vistas
asp-controller, asp-action	Genera la URL de un enlace o formulario	Enlace "Editar" de cada fila
asp-route-{parámetro}	Agrega valores a la URL	asp-route-id para persona; asp-route-idProf y asp-route-ccPer

Tag Helper	Qué hace	Ejemplo de uso en mis vistas
		para estudio
asp-for (input, label)	Genera id, name, tipo de campo, texto de la etiqueta y atributos de validación	Campos nombre, apellido, edad
asp-validation-for	Muestra el error de un campo	Debajo de cada campo
asp-validation-summary	Muestra errores generales del formulario	Error "la cédula ya existe"
select asp-for asp-items	Lista desplegable con la opción actual marcada	Persona y profesión en Estudio; dueño en Teléfono

El Tag Helper de formulario agrega por sí solo el token antifalsificación oculto, que la acción POST valida.

Vistas parciales

Una parcial es un pedazo de vista reutilizable. Uso la de validación en Create y Edit, y puedo crear _PersonaForm.cshtml para no repetir los mismos campos en Create y Edit.

Estilos

La plantilla trae Bootstrap 5: tablas con la clase table, botones btn btn-primary y formularios con form-control. No hace falta CSS propio para cumplir el laboratorio.

Formularios, validación y seguridad

Un formulario viaja en cuatro pasos: el navegador valida, el servidor enlaza los datos al modelo, valida de nuevo y solo entonces llama al repositorio.

Model binding (enlace de modelos)

ASP.NET Core toma los campos del formulario, la ruta y la cadena de consulta, y los asigna por nombre a los parámetros de la acción o a las propiedades del modelo. Si un dato no se puede convertir (por ejemplo "abc" en edad), queda registrado como error en ModelState.

Validación

- En el servidor: la acción revisa ModelState.IsValid antes de guardar. Si es falso, devuelve la misma vista con el modelo, y los Tag Helpers pintan los errores junto a cada campo.
- En el navegador: los Tag Helpers generan atributos data-val-* y jQuery Validation Unobtrusive los aplica sin recargar la página. Se activa incluyendo _ValidationScriptsPartial en Create y Edit.
- La del navegador es comodidad; la del servidor es la que protege, porque la primera se puede saltar.

Reglas de validación de mi parte

Las entidades generadas por scaffolding no traen DataAnnotations; las reglas salen del modelo de datos. Las pongo en clases de metadatos o en ViewModels para no editar los archivos generados, que se sobrescriben si se vuelve a ejecutar Scaffold-DbContext.

Entidad	Campo	Regla
Persona	cc	Obligatoria, número positivo, no repetida
Persona	nombre, apellido	Obligatorios, máximo 45 caracteres
Persona	genero	Obligatorio, solo M o F (lista de dos opciones, no texto libre)
Persona	edad	Opcional, rango razonable (0 a 120)
Profesion	id	Obligatorio, no repetido
Profesion	nom	Obligatorio, máximo 90 caracteres
Estudio	persona y profesión	Obligatorias, elegidas de lista; la pareja no puede repetirse
Estudio	univer	Opcional, máximo 50 caracteres
Estudio	fecha	Opcional, no futura
Telefono	num	Obligatorio, máximo 15 caracteres, no repetido
Telefono	oper	Obligatorio, máximo 45 caracteres
Telefono	dueño	Obligatorio, elegido de lista

Con .NET 10 los tipos de referencia anulables están activos: una propiedad string no anulable ya se trata como obligatoria. El scaffolding marca como string? las columnas que admiten nulos, así que la obligatoriedad básica viene dada.

Antifalsificación (CSRF)

El formulario incluye un token oculto y cada acción POST lleva [ValidateAntiForgeryToken]. Así otro sitio no puede enviar formularios en nombre del usuario.

Sobrepublicación (overposting)

Si la acción recibe la entidad completa, alguien podría enviar campos que el formulario no muestra. Dos defensas que documenta Microsoft:

- [Bind] con la lista de campos permitidos en Create.
- En Edit, leer la entidad existente y actualizar solo los campos permitidos (TryUpdateModelAsync), o usar un ViewModel con solo esos campos.

En Edit de Persona, la cédula no se puede cambiar porque es la llave: se muestra de solo lectura y no se acepta del formulario.

Errores de base de datos

Si el repositorio falla al guardar (por ejemplo, cédula repetida), la acción captura la excepción y agrega un mensaje con ModelState.AddModelError, en lugar de mostrar una página de error. Si al borrar falla, redirige a la confirmación de Delete con el mensaje.

Casos especiales del modelo

Persona y Profesion son CRUD directos; Estudio y Telefono tienen detalles que hay que resolver antes de programar.

Estudio: llave compuesta

- Se identifica por la pareja (idProf, ccPer), no por un id. Todos los enlaces de la lista llevan los dos valores, y las acciones Details, Edit y Delete reciben los dos parámetros.
- En Create, persona y profesión se eligen de listas desplegables. En Edit se muestran de solo lectura: cambiarlas sería cambiar la llave, es decir, otro registro.
- Si la pareja ya existe, el repositorio falla; muestro "Esta persona ya tiene registrada esa profesión".

Listas desplegables

- Estudio necesita la lista de personas (valor cc, texto nombre y apellido) y la de profesiones (valor id, texto nom). Teléfono necesita la de personas para el dueño.
- El controlador las arma con los repositorios de Persona y Profesion, así que EstudioController recibe tres interfaces y TelefonoController dos.
- Microsoft recomienda pasar las listas en un ViewModel en vez de ViewBag; con ViewBag funciona pero sin tipado. Decisión: ViewModels EstudioFormViewModel y TelefonoFormViewModel.
- Si el formulario vuelve con errores, hay que volver a cargar las listas antes de devolver la vista, o aparecen vacías.

Género

Como la columna solo acepta M o F, se elige de una lista con dos opciones (Masculino → M, Femenino → F). En Index y Details se muestra el texto completo, no la letra.

Detalle de Persona

Details muestra los datos de la persona y, debajo, dos tablas: sus estudios (profesión, universidad, fecha) y sus teléfonos (número, operador). Es la vista que mejor demuestra las relaciones del modelo.

Borrados con dependientes

- Persona: con CASCADE en la base, borrar una persona borra sus estudios y teléfonos. La confirmación de Delete debe advertirlo y mostrar cuántos tiene.
- Profesion: igual, borra los estudios que la usan.
- Si el equipo decide no usar CASCADE, la base rechaza el borrado y muestro "No se puede borrar: tiene estudios o teléfonos asociados".

Nombres en lugar de llaves

Index de Estudio muestra "Ana Pérez — Ingeniería de Sistemas" en lugar de "1012345678 — 3". Index de Teléfono muestra el nombre del dueño. Para eso necesito que el repositorio cargue las relaciones.

Inventario de controladores y vistas

Son 4 controladores, 32 acciones y 20 vistas. Cada fila es una acción.

Controlador	Acción	Verbo	Ruta	Vista	Método del repositorio
PersonaController	Index	GET	/Persona	Index	Listar todos
PersonaController	Details	GET	/Persona/Details/{cc}	Details	Buscar por cc, con estudios y teléfonos
PersonaController	Create	GET	/Persona/Create	Create	—
PersonaController	Create	POST	/Persona/Create	Index (redirección)	Crear
PersonaController	Edit	GET	/Persona/Edit/{cc}	Edit	Buscar por cc
PersonaController	Edit	POST	/Persona/Edit/{cc}	Index (redirección)	Actualizar
PersonaController	Delete	GET	/Persona/Delete/{cc}	Delete	Buscar por cc
PersonaController	DeleteConfirmed	POST	/Persona/Delete/{cc}	Index (redirección)	Eliminar
ProfesionController	Las mismas 8 acciones	GET y POST	/Profesion/.../{id}	Index, Details, Create, Edit, Delete	Las mismas 5 operaciones
EstudioController	Las mismas 8 acciones	GET y POST	/Estudio/...?idProf=&ccPer=	Index, Details, Create, Edit, Delete	Las 5 operaciones + listar personas y profesiones
TelefonoController	Las mismas 8 acciones	GET y POST	/Telefono/.../{num}	Index, Details, Create, Edit, Delete	Las 5 operaciones + listar personas

Además: _Layout.cshtml con el menú (Personas, Profesiones, Estudios, Teléfonos, Swagger) y Home/Index como página de inicio con acceso a las cuatro listas.

Plan de trabajo

Empiezo por Persona y Profesion, que no dependen de nadie, y dejo Estudio y Teléfono para el final porque necesitan listas desplegables.

1. Reunión de los tres: acordar arquitectura y firmas DAO (sección Organización).
2. Tener el ambiente: Visual Studio 2026, SQL Server 2025 Express, SSMS 22 y el repositorio clonado.
3. Cuando el integrante 1 suba la base a main, crear mi rama feature/mvc a partir de main.
4. Verificar que la app arranque y que el Explorador de objetos de SQL Server vea persona_db con datos.
5. PersonaController y sus 5 vistas. Atajo: en Visual Studio, Agregar > Controlador > "Controlador de MVC con vistas que usa Entity Framework" genera el controlador y las 5 vistas. Ese controlador usa el DbContext directo, lo que rompe el patrón DAO; hay que reemplazarlo por IPersonaRepository y conservar las vistas.
6. Ajustar las vistas de Persona: género como lista, cédula de solo lectura en Edit, textos en español, estudios y teléfonos en Details.
7. ProfesionController y sus 5 vistas, igual que Persona.
8. EstudioController: ViewModel con listas, llave compuesta en rutas y enlaces, mensaje de pareja repetida.
9. TelefonoController: ViewModel con lista de dueños.
10. Menú en _Layout y página de inicio.
11. Validaciones y mensajes (sección Formularios), luego mi plan de pruebas.
12. Pull request de feature/mvc a main; un compañero lo revisa.
13. Pruebas finales con los tres, capturas de las vistas y aporte a conclusiones.

Plan de pruebas de la interfaz

Antes del pull request repito estos casos en las cuatro entidades; los marcados con Estudio o Teléfono aplican solo ahí.

#	Caso	Pasos	Resultado esperado
1	Listar	Abrir /Persona	Tabla con los datos del DML y enlaces Detalles, Editar, Borrar
2	Crear válido	Llenar el formulario y guardar	Redirige a la lista con mensaje de éxito; el registro aparece
3	Crear vacío	Guardar sin llenar campos	Errores junto a cada campo obligatorio, sin recargar la página
4	Crear sin JavaScript	Desactivar JavaScript y guardar vacío	El servidor devuelve el formulario con los mismos errores
5	Llave repetida	Crear una persona con una cédula existente	Mensaje "La cédula ya existe"; no se pierde lo escrito

#	Caso	Pasos	Resultado esperado
6	Longitud máxima	Escribir 46 caracteres en nombre	Error de longitud
7	Editar	Cambiar el apellido y guardar	Lista muestra el cambio; la cédula no se pudo editar
8	No existe	Abrir /Persona/Details/999	Página 404, no error de servidor
9	Borrar	Borrar una persona con estudios	Confirmación avisa los dependientes; luego desaparecen de las listas
10	Recargar tras guardar	Crear y presionar F5 en la lista	No se crea un duplicado (PRG)
11	Listas (Estudio, Teléfono)	Abrir Create	Listas con nombres de personas y profesiones
12	Error con listas (Estudio, Teléfono)	Guardar con un error	El formulario vuelve con las listas llenas
13	Pareja repetida (Estudio)	Crear un estudio que ya existe	Mensaje de pareja repetida
14	Coherencia con la API	Crear en la vista y consultar en Swagger	El registro aparece en GET /api/personas
15	Navegación	Usar todos los enlaces del menú	Ningún enlace roto

Referencias

Documentación oficial de Microsoft para ASP.NET Core 10.0.

- Microsoft. Información general de ASP.NET Core MVC.
- Microsoft. Enrutamiento a acciones de controlador.
- Microsoft. Vistas en ASP.NET Core MVC.
- Microsoft. Diseño (layout) en ASP.NET Core.
- Microsoft. Tag Helpers en formularios.
- Microsoft. Enlace de modelos.
- Microsoft. Validación de modelos.
- Microsoft. Prevención de ataques CSRF.
- Microsoft. Inserción de dependencias en ASP.NET Core.
- Microsoft. Tutorial: CRUD con EF Core en una app MVC.
- Microsoft. Introducción a ASP.NET Core MVC.